

```
In [3]: import os
import shap
import numpy as np
import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt

import xgboost as xgb
from xgboost import XGBRegressor
from xgboost import plot_tree
from xgboost import plot_importance
from xgboost.callback import EarlyStopping

from sklearn.model_selection import train_test_split
from sklearn.metrics import mean_squared_error, mean_absolute_error, r2_score
from sklearn.preprocessing import StandardScaler, MinMaxScaler
from sklearn.linear_model import LinearRegression
from sklearn.ensemble import RandomForestRegressor

import statsmodels.api as sm
from statsmodels.stats.outliers_influence import variance_inflation_factor

import scipy.stats as stats

import warnings
warnings.filterwarnings('ignore')

# ===== UPLOAD Data =====

df_up = pd.read_csv('Modeling Test Data.csv')

print (df_up.head())
print (df_up.shape)
print(df_up.info())

print(df_up.describe())
```

	y	var_1	var_2	var_3	var_4	var_5 \
0	58.891377	5285.076955	8102.610429	-142802.4627	0.731818	61045.686450
1	57.641838	2094.137183	7432.119245	-147434.0657	2.527586	48485.220290
2	53.746536	5830.245294	6613.941822	-129496.5961	2.625691	36301.747910
3	36.168986	13039.246420	5270.829420	-439972.8126	0.844833	-5909.877785
4	32.669772	4611.755026	3456.822275	-138594.7123	4.058533	37602.705370

	var_6	var_7	var_8	var_9	var_10
0	183502.7911	2418498.392	27880.60083	360626.7592	4
1	232809.7287	1779351.332	32153.75776	382670.3624	5
2	214837.0432	2072100.771	30642.70182	370120.6062	2
3	173613.1204	1899841.158	41311.52728	329891.8986	4
4	228498.4096	1925463.972	27522.27266	473752.7977	3

(10000, 11)

<class 'pandas.core.frame.DataFrame'>

RangeIndex: 10000 entries, 0 to 9999

Data columns (total 11 columns):

#	Column	Non-Null Count	Dtype
0	y	10000 non-null	float64
1	var_1	10000 non-null	float64
2	var_2	10000 non-null	float64
3	var_3	10000 non-null	float64
4	var_4	10000 non-null	float64
5	var_5	10000 non-null	float64
6	var_6	10000 non-null	float64
7	var_7	10000 non-null	float64
8	var_8	10000 non-null	float64
9	var_9	10000 non-null	float64
10	var_10	10000 non-null	int64

dtypes: float64(10), int64(1)

memory usage: 859.5 KB

None

	y	var_1	var_2	var_3	var_4 \
count	10000.000000	10000.000000	10000.000000	10000.000000	10000.000000
mean	12.190351	8445.963010	3450.252510	168505.815510	1.610510
std	16.020137	3540.407793	2544.552718	154606.758685	0.799212
min	-20.486397	-5583.153749	-6338.250221	-439972.812600	-1.328607
25%	0.342919	6012.666381	1689.792306	64840.482335	1.070263
50%	10.881019	8463.373285	3430.595021	167029.781400	1.598712
75%	23.047831	10864.843605	5165.408836	272343.174250	2.146489
max	94.749833	23299.976630	12288.699000	857156.728100	4.639956

	var_5	var_6	var_7	var_8 \
count	10000.000000	10000.000000	1.000000e+04	10000.000000
mean	73390.402240	85873.402570	1.150636e+06	22190.066240
std	42488.751840	71600.020020	8.840513e+05	9835.270071
min	-77050.448580	-162312.422300	-2.692945e+06	-12674.425590
25%	44707.280773	37958.405845	5.353268e+05	15593.819880
50%	73543.761555	86582.980525	1.149984e+06	22091.102400
75%	102189.829900	133951.621325	1.742196e+06	28924.445840
max	242913.447400	357758.526300	4.315470e+06	58807.189120

	var_9	var_10
count	10000.000000	10000.000000
mean	153862.395490	3.032500

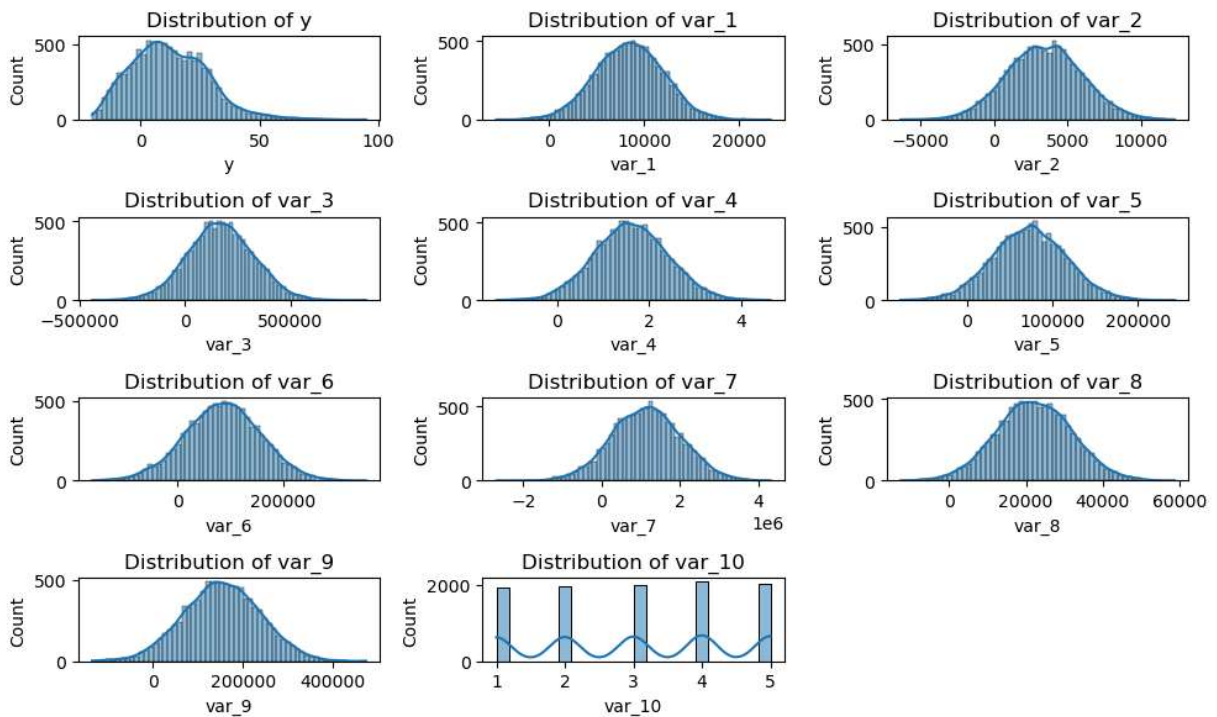
std	84665.459925	1.410901
min	-135125.465700	1.000000
25%	96818.711418	2.000000
50%	154089.232450	3.000000
75%	211875.685575	4.000000
max	473752.797700	5.000000

In [5]: # ===== Check Original Data Distributions =====

```
df = df_up.copy()

plt.figure(figsize=(10, 6))
for i, col in enumerate(df.columns, 1):
    plt.subplot(4, 3, i) # Adjust grid if needed (you have 11 columns)
    sns.histplot(df[col], kde=True)
    plt.title(f'Distribution of {col}')
    plt.tight_layout()

plt.show()
```



In [7]: # ===== Create Dummy Variables and Interaction Terms and Check Their Re

```
df = df_up.copy()
dummies = pd.get_dummies(df['var_10'], prefix='var_10').astype(int)
df = pd.concat([df, dummies], axis=1)
print('Dummy Variable Mean:', dummies.mean())

df = df.drop(columns='var_10_5')
x_dums = dummies.columns.drop('var_10_5').tolist()

interactions = {}
interactions['var_2*var_2'] = df['var_2'] * df['var_2']
interactions['var_3*var_3'] = df['var_3'] * df['var_3']
```

```

interactions['var_9*var_9']=df['var_9']*df['var_9']

interactions['var3_below_0'] = np.minimum(df['var_3'], 0)
interactions['var3_0_to_200k'] = np.minimum(np.maximum(df['var_3'], 0), 200000)
interactions['var3_gt_200k'] = np.maximum(df['var_3'] - 200000, 0)

interactions['var9_below_0'] = np.minimum(df['var_9'], 0)
interactions['var9_0_to_100k'] = np.minimum(np.maximum(df['var_9'], 0), 100000)
interactions['var9_100k_to_200k'] = np.minimum(np.maximum(df['var_9']-100000, 0), 1)
interactions['var9_above_200k'] = np.maximum(df['var_9'] - 200000, 0)

df = pd.concat([df, pd.DataFrame(interactions)], axis=1)

n_cols = 3 # 3 plots per row
n_rows = (len(df.columns) + n_cols - 1) // n_cols # Auto calculate rows
plt.figure(figsize=(n_cols * 6, n_rows * 4.5)) # Bigger figure size

for i, col in enumerate(df.columns, 1):
    plt.subplot(n_rows, n_cols, i)
    sns.scatterplot(x=df[col], y=df['y'], alpha=0.6)
    sns.regplot(x=df[col], y=df['y'], scatter=False, color='red', line_kws={'linewi
    plt.title(f'{col} vs y')
    plt.xlabel(col, fontsize=10)
    plt.ylabel('Frequency', fontsize=10)

plt.tight_layout(pad=2.0) # Important: adjust spacing
plt.show()

x_scale=df_up.columns.drop(['y', 'var_4', 'var_10']).tolist()

x_scale_p=['var3_below_0', 'var3_0_to_200k', 'var3_gt_200k', 'var9_below_0', 'var9_
x_inter=['var_2*var_2', 'var_3*var_3', 'var_9*var_9']

x_nscale=['var_4', 'var_10']+x_dums

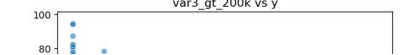
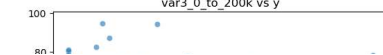
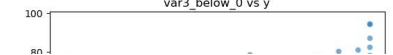
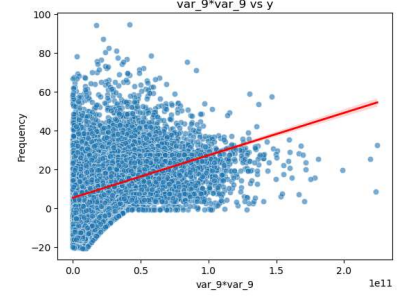
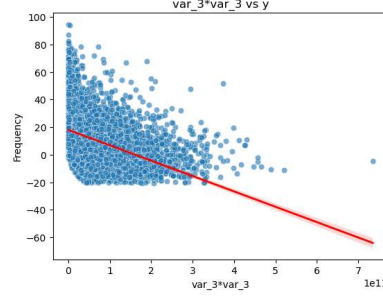
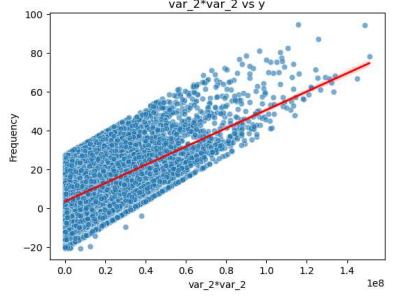
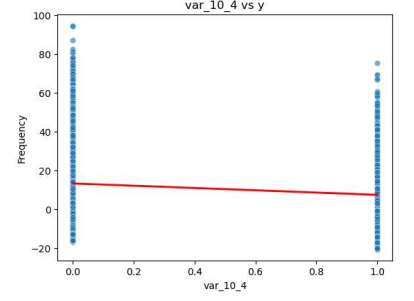
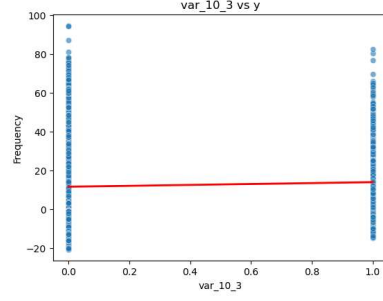
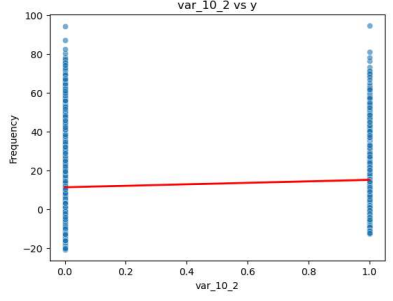
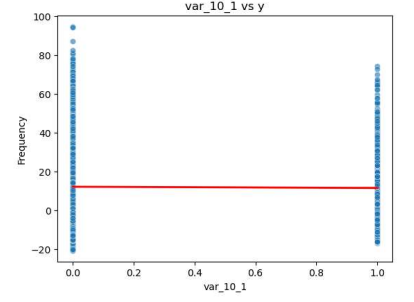
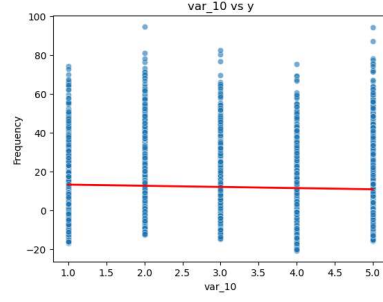
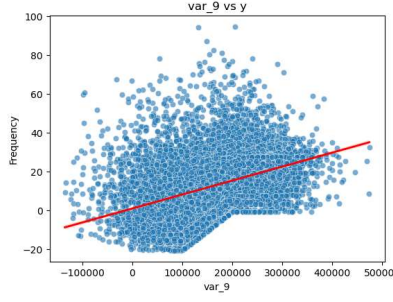
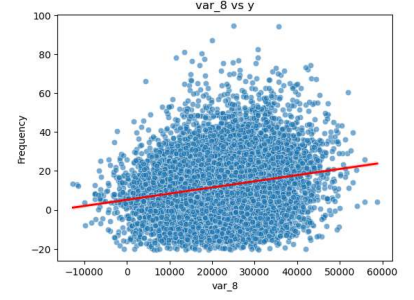
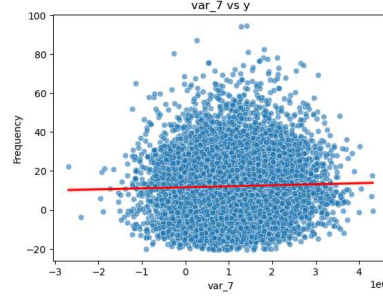
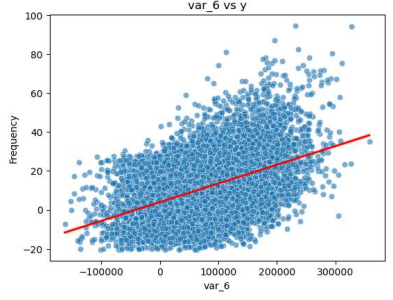
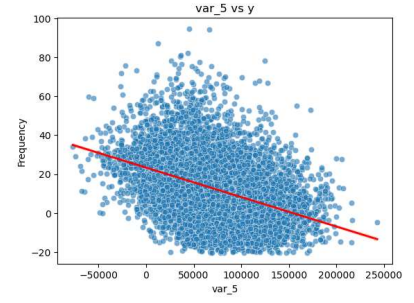
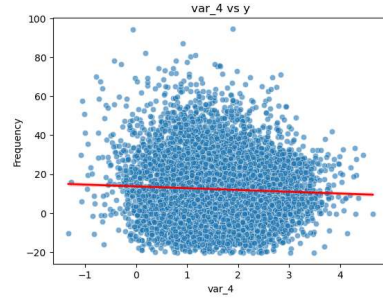
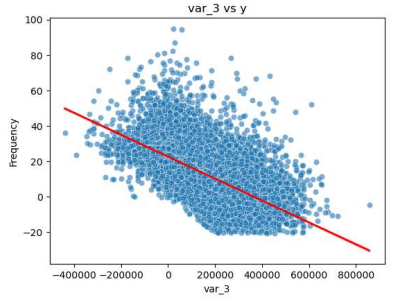
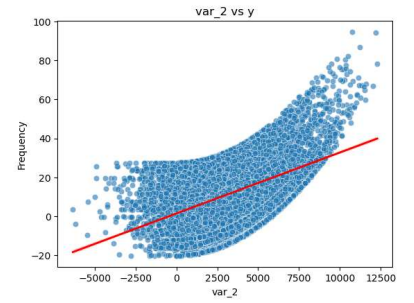
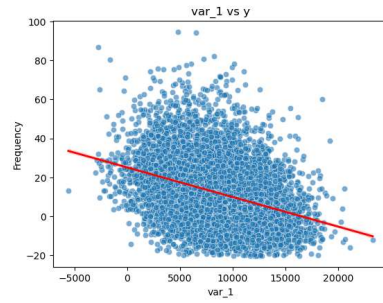
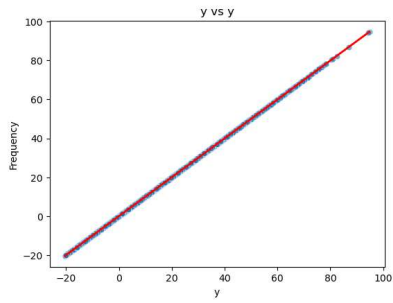
print('Scale Variables:', x_scale)
print('Non Scale Variables:', x_nscale)
print('Dummy Variables:', x_dums)
print('Data Shape:', df.shape)

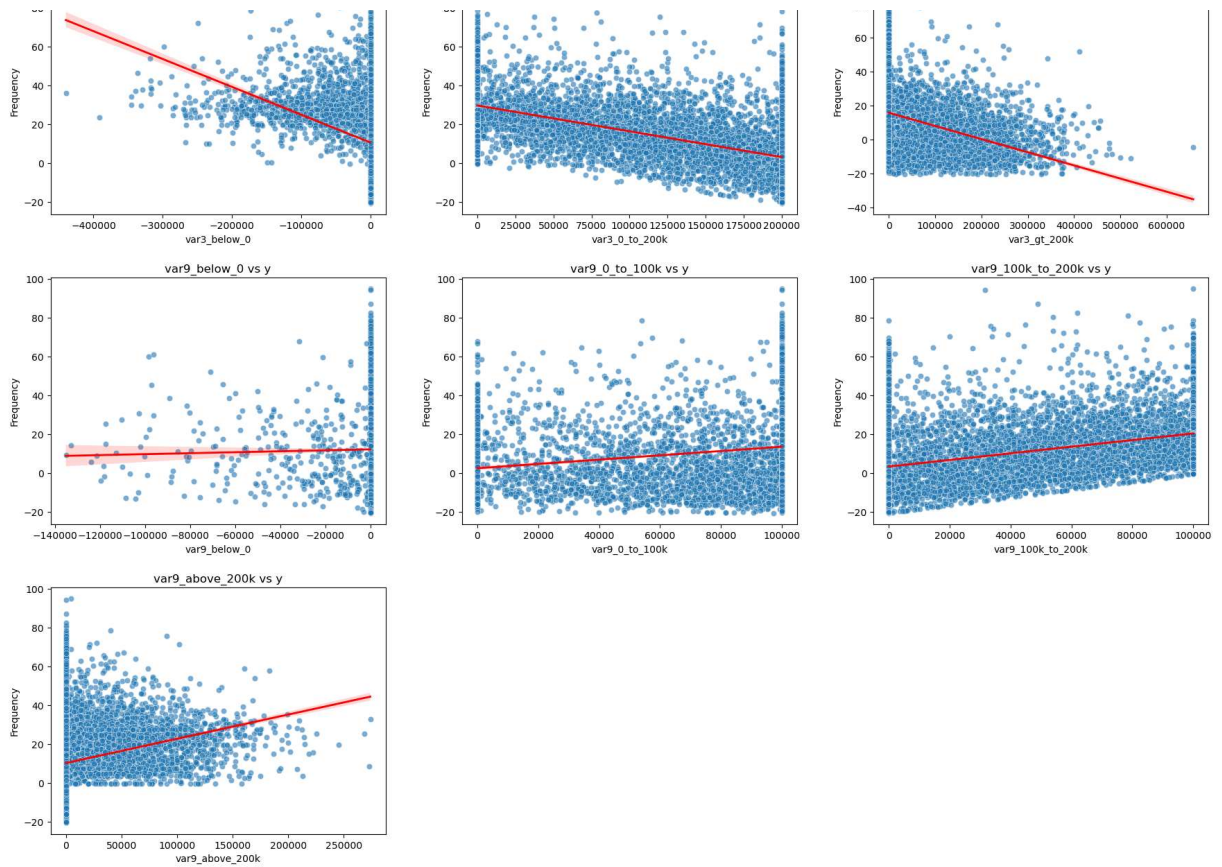
```

```

Dummy Variable Mean: var_10_1    0.1935
var_10_2    0.1957
var_10_3    0.1989
var_10_4    0.2086
var_10_5    0.2033
dtype: float64

```





Scale Variables: ['var_1', 'var_2', 'var_3', 'var_5', 'var_6', 'var_7', 'var_8', 'var_9']

Non Scale Variables: ['var_4', 'var_10', 'var_10_1', 'var_10_2', 'var_10_3', 'var_10_4']

Dummy Variables: ['var_10_1', 'var_10_2', 'var_10_3', 'var_10_4']

Data Shape: (10000, 25)

In [11]: # ===== Calculate Data Correlations =====

```
corr = df[df.columns].corr()
print(corr)
```

	y	var_1	var_2	var_3	var_4	var_5	\
y	1.000000	-0.336184	0.497815	-0.596058	-0.045849	-0.401891	
var_1	-0.336184	1.000000	-0.135317	0.196789	-0.251892	0.191826	
var_2	0.497815	-0.135317	1.000000	0.033808	-0.359648	-0.035389	
var_3	-0.596058	0.196789	0.033808	1.000000	0.036211	0.808120	
var_4	-0.045849	-0.251892	-0.359648	0.036211	1.000000	-0.075834	
var_5	-0.401891	0.191826	-0.035389	0.808120	-0.075834	1.000000	
var_6	0.429807	-0.169298	0.602046	0.040761	-0.188209	0.277714	
var_7	0.029095	0.469718	-0.026948	0.225848	0.075821	0.303735	
var_8	0.194788	0.212547	0.208451	-0.283731	-0.039344	-0.413344	
var_9	0.379986	-0.342938	-0.370873	-0.409241	0.474635	-0.054084	
var_10	-0.052502	0.016748	0.001143	-0.005825	-0.003559	-0.009498	
var_10_1	-0.015661	-0.022103	-0.000960	-0.000652	0.001820	0.008250	
var_10_2	0.094442	0.000040	-0.001616	0.015607	0.002726	0.005567	
var_10_3	0.058478	0.013284	0.004849	-0.015254	-0.010083	-0.011981	
var_10_4	-0.148343	0.001588	-0.005031	0.007015	0.017108	0.008126	
var_2*var_2	0.583841	-0.120922	0.890143	0.024499	-0.326090	-0.043653	
var_3*var_3	-0.435278	0.163994	0.033033	0.838819	0.031845	0.675880	
var_9*var_9	0.378915	-0.326541	-0.340495	-0.383578	0.446972	-0.055403	
var3_below_0	-0.329417	0.116831	0.015588	0.572810	0.020174	0.467282	
var3_0_to_200k	-0.637668	0.174429	0.027745	0.885916	0.032290	0.716766	
var3_gt_200k	-0.387252	0.158913	0.031511	0.817610	0.029666	0.657894	
var9_below_0	0.012973	-0.104450	-0.137680	-0.139382	0.154756	-0.009757	
var9_0_to_100k	0.183897	-0.241753	-0.283723	-0.297897	0.344664	-0.033949	
var9_100k_to_200k	0.443639	-0.313132	-0.330705	-0.367469	0.430039	-0.046309	
var9_above_200k	0.259719	-0.259699	-0.266779	-0.306455	0.353010	-0.049825	

	var_6	var_7	var_8	var_9	...	var_2*var_2	\
y	0.429807	0.029095	0.194788	0.379986	...	0.583841	
var_1	-0.169298	0.469718	0.212547	-0.342938	...	-0.120922	
var_2	0.602046	-0.026948	0.208451	-0.370873	...	0.890143	
var_3	0.040761	0.225848	-0.283731	-0.409241	...	0.024499	
var_4	-0.188209	0.075821	-0.039344	0.474635	...	-0.326090	
var_5	0.277714	0.303735	-0.413344	-0.054084	...	-0.043653	
var_6	1.000000	0.046405	0.103821	0.097726	...	0.524203	
var_7	0.046405	1.000000	-0.011027	0.305034	...	-0.026781	
var_8	0.103821	-0.011027	1.000000	-0.146210	...	0.190180	
var_9	0.097726	0.305034	-0.146210	1.000000	...	-0.337852	
var_10	-0.009607	0.006891	0.015578	-0.011332	...	0.006334	
var_10_1	0.010129	-0.011182	-0.010303	0.014737	...	-0.006615	
var_10_2	-0.009323	-0.002892	-0.009936	-0.013066	...	0.001014	
var_10_3	0.007322	0.011456	0.007621	0.005809	...	0.000303	
var_10_4	0.006891	0.005521	0.000112	0.008897	...	0.000167	
var_2*var_2	0.524203	-0.026781	0.190180	-0.337852	...	1.000000	
var_3*var_3	0.035863	0.192050	-0.235829	-0.344661	...	0.026365	
var_9*var_9	0.094097	0.280623	-0.138104	0.932485	...	-0.288631	
var3_below_0	0.025495	0.132206	-0.163701	-0.229004	...	0.009584	
var3_0_to_200k	0.035514	0.196110	-0.252696	-0.363666	...	0.017909	
var3_gt_200k	0.032910	0.187175	-0.230153	-0.336026	...	0.025740	
var9_below_0	0.033275	0.120201	-0.035164	0.352694	...	-0.158667	
var9_0_to_100k	0.063857	0.220976	-0.106187	0.730826	...	-0.293797	
var9_100k_to_200k	0.091503	0.279988	-0.137994	0.904636	...	-0.301263	
var9_above_200k	0.074298	0.217548	-0.104979	0.735084	...	-0.206465	

	var_3*var_3	var_9*var_9	var3_below_0	var3_0_to_200k	\
y	-0.435278	0.378915	-0.329417	-0.637668	

var_1	0.163994	-0.326541	0.116831	0.174429
var_2	0.033033	-0.340495	0.015588	0.027745
var_3	0.838819	-0.383578	0.572810	0.885916
var_4	0.031845	0.446972	0.020174	0.032290
var_5	0.675880	-0.055403	0.467282	0.716766
var_6	0.035863	0.094097	0.025495	0.035514
var_7	0.192050	0.280623	0.132206	0.196110
var_8	-0.235829	-0.138104	-0.163701	-0.252696
var_9	-0.344661	0.932485	-0.229004	-0.363666
var_10	-0.013029	-0.009390	0.012823	-0.006321
var_10_1	0.010368	0.010309	-0.018777	-0.001861
var_10_2	0.012593	-0.013630	0.005917	0.015384
var_10_3	-0.015699	0.012057	-0.000531	-0.014274
var_10_4	-0.001123	0.008750	0.012211	0.012166
var_2*var_2	0.026365	-0.288631	0.009584	0.017909
var_3*var_3	1.000000	-0.291274	0.122446	0.612132
var_9*var_9	-0.291274	1.000000	-0.252655	-0.354999
var3_below_0	0.122446	-0.252655	1.000000	0.494661
var3_0_to_200k	0.612132	-0.354999	0.494661	1.000000
var3_gt_200k	0.977285	-0.283788	0.170623	0.522893
var9_below_0	-0.159981	0.128539	-0.037282	-0.097336
var9_0_to_100k	-0.300463	0.478950	-0.112848	-0.238185
var9_100k_to_200k	-0.314275	0.836936	-0.191806	-0.330634
var9_above_200k	-0.201826	0.905749	-0.241981	-0.295060

	var3_gt_200k	var9_below_0	var9_0_to_100k	\
y	-0.387252	0.012973	0.183897	
var_1	0.158913	-0.104450	-0.241753	
var_2	0.031511	-0.137680	-0.283723	
var_3	0.817610	-0.139382	-0.297897	
var_4	0.029666	0.154756	0.344664	
var_5	0.657894	-0.009757	-0.033949	
var_6	0.032910	0.033275	0.063857	
var_7	0.187175	0.120201	0.220976	
var_8	-0.230153	-0.035164	-0.106187	
var_9	-0.336026	0.352694	0.730826	
var_10	-0.011102	-0.007294	-0.009726	
var_10_1	0.009197	0.014421	0.017796	
var_10_2	0.012653	-0.001487	-0.008062	
var_10_3	-0.015526	-0.012686	-0.010076	
var_10_4	-0.003779	-0.001475	0.007977	
var_2*var_2	0.025740	-0.158667	-0.293797	
var_3*var_3	0.977285	-0.159981	-0.300463	
var_9*var_9	-0.283788	0.128539	0.478950	
var3_below_0	0.170623	-0.037282	-0.112848	
var3_0_to_200k	0.522893	-0.097336	-0.238185	
var3_gt_200k	1.000000	-0.158795	-0.295025	
var9_below_0	-0.158795	1.000000	0.455704	
var9_0_to_100k	-0.295025	0.455704	1.000000	
var9_100k_to_200k	-0.304165	0.171973	0.579306	
var9_above_200k	-0.197189	0.064738	0.218076	

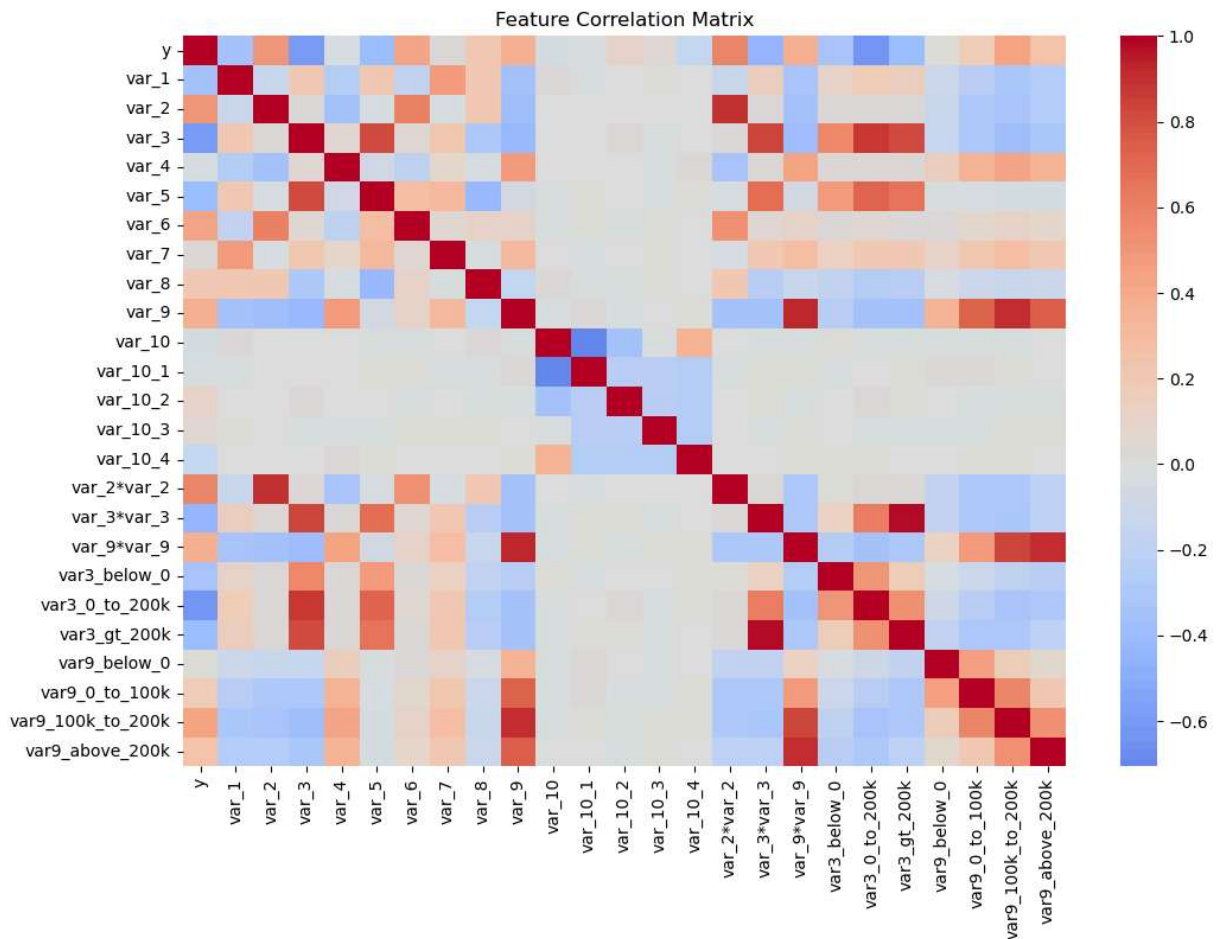
	var9_100k_to_200k	var9_above_200k
y	0.443639	0.259719
var_1	-0.313132	-0.259699
var_2	-0.330705	-0.266779

var_3	-0.367469	-0.306455
var_4	0.430039	0.353010
var_5	-0.046309	-0.049825
var_6	0.091503	0.074298
var_7	0.279988	0.217548
var_8	-0.137994	-0.104979
var_9	0.904636	0.735084
var_10	-0.011073	-0.005321
var_10_1	0.009956	0.007126
var_10_2	-0.010543	-0.013172
var_10_3	0.011022	0.012194
var_10_4	0.008957	0.005389
var_2*var_2	-0.301263	-0.206465
var_3*var_3	-0.314275	-0.201826
var_9*var_9	0.836936	0.905749
var3_below_0	-0.191806	-0.241981
var3_0_to_200k	-0.330634	-0.295060
var3_gt_200k	-0.304165	-0.197189
var9_below_0	0.171973	0.064738
var9_0_to_100k	0.579306	0.218076
var9_100k_to_200k	1.000000	0.540160
var9_above_200k	0.540160	1.000000

[25 rows x 25 columns]

```
In [13]: # ===== Correlations Plot=====

plt.figure(figsize=(12,8))
sns.heatmap(corr, cmap="coolwarm", center=0)
plt.title("Feature Correlation Matrix")
plt.show()
```



```
In [15]: # ===== Data Sets Split and Scaling =====

X_train, X_temp, y_train, y_temp = train_test_split(df[x_scale+x_scale_p+x_nscale],
X_val, X_test, y_val, y_test = train_test_split(X_temp, y_temp, test_size=0.50,

scaler = StandardScaler()
X_train_scaled = pd.DataFrame(scaler.fit_transform(X_train[x_scale]), columns=x_sca
X_val_scaled = pd.DataFrame(scaler.transform(X_val[x_scale]), columns=x_scale, inde
X_test_scaled = pd.DataFrame(scaler.transform(X_test[x_scale]), columns=x_scale, in

minmax = MinMaxScaler(feature_range=(0, 1))
X_train_pieces = pd.DataFrame(minmax.fit_transform(X_train[x_scale_p]), columns=x_s
X_val_pieces = pd.DataFrame(minmax.transform(X_val[x_scale_p]), columns=x_scale_p,
X_test_pieces = pd.DataFrame(minmax.transform(X_test[x_scale_p]), columns=x_scale_p

X_train_data = pd.concat([X_train_scaled, X_train_pieces, X_train[x_nscale]], axis=
X_val_data = pd.concat([X_val_scaled, X_val_pieces, X_val[x_nscale]], axis=1)
X_test_data = pd.concat([X_test_scaled, X_test_pieces, X_test[x_nscale]], axis=1)

interactions_train = {}
interactions_val = {}
interactions_test = {}
for col in x_inter:
    col1=col.split('*')[0].strip()
    col2=col.split('*')[1].strip()
    interactions_train[col]=X_train_data[col1]*X_train_data[col2]
```

```
interactions_val[col]=X_val_data[col1]*X_val_data[col2]
interactions_test[col]=X_test_data[col1]*X_test_data[col2]
```

```
X_train_data = pd.concat([X_train_data, pd.DataFrame(interactions_train)], axis=1)
X_val_data = pd.concat([X_val_data, pd.DataFrame(interactions_val)], axis=1)
X_test_data = pd.concat([X_test_data, pd.DataFrame(interactions_test)], axis=1)
```

In [17]: # ===== Model Evaluation Function For Linear Regression Models =====

```
def evaluate_model(model, X_train, y_train, X_val, y_val, X_test, y_test, name="Model"):
    y_train_pred = model.predict(X_train)
    X_val_const = sm.add_constant(X_val)
    y_val_pred = model.predict(X_val_const)
    X_test_const = sm.add_constant(X_test)
    y_test_pred = model.predict(X_test_const)

    print(f"\n=== {name} ===")
    print(f"Train R² : {r2_score(y_train, y_train_pred):.4f}")
    print(f"Val R² : {r2_score(y_val, y_val_pred):.4f}")
    print(f"Test R² : {r2_score(y_test, y_test_pred):.4f}")
    print(f"Train MAE: {mean_absolute_error(y_train, y_train_pred):.4f}")
    print(f"Val MAE: {mean_absolute_error(y_val, y_val_pred):.4f}")
    print(f"Test MAE: {mean_absolute_error(y_test, y_test_pred):.4f}")
    print(f"Train RMSE: {np.sqrt(mean_squared_error(y_train, y_train_pred)):.4f}")
    print(f"Val RMSE: {np.sqrt(mean_squared_error(y_val, y_val_pred)):.4f}")
    print(f"Test RMSE: {np.sqrt(mean_squared_error(y_test, y_test_pred)):.4f}")

    residuals = y_train - y_train_pred

    # Basic statistics
    (osm, osr), (slope, intercept, r) = stats.probplot(residuals, dist="norm")
    print(r, slope, intercept)

    print("Mean of Residuals:", round(residuals.mean(), 4))
    print("Std of Residuals:", round(residuals.std(), 4))
```

In [19]: # ===== VIF Calculation Function =====

```
def get_VIF (X_train):
    vif_data = pd.DataFrame()
    vif_data['Variable'] = X_train.columns
    vif_data['VIF'] = [variance_inflation_factor(X_train, i)
                       for i in range(X_train.shape[1])]

    vif_data = vif_data.sort_values('VIF', ascending=False)
    print(vif_data.round(2))
```

In [21]: # ===== Linear Regression Model =====

```
x_var=['var_2', 'var_3', 'var_9']

# Add constant (intercept)
X_train_const = sm.add_constant(X_train_data[x_var])
```

```
# Fit model
model_ols = sm.OLS(y_train, X_train_const).fit()

# Show summary
print(model_ols.summary())

# Show summary VIF
get_VIF (X_train_const)
```

```

                                OLS Regression Results
=====
Dep. Variable:                  y      R-squared:                0.772
Model:                        OLS      Adj. R-squared:           0.772
Method:                    Least Squares  F-statistic:              7881.
Date:                Tue, 09 Jun 2026  Prob (F-statistic):        0.00
Time:                  11:31:19      Log-Likelihood:          -24112.
No. Observations:                7000      AIC:                    4.823e+04
Df Residuals:                    6996      BIC:                    4.826e+04
Df Model:                          3
Covariance Type:                nonrobust
=====
               coef      std err          t      P>|t|      [0.025      0.975]
-----
const          12.0205      0.091     132.628      0.000      11.843      12.198
var_2          10.7471      0.098     109.477      0.000      10.555      10.940
var_3          -6.9185      0.100    -68.863      0.000      -7.115      -6.722
var_9           7.2176      0.108     66.951      0.000       7.006       7.429
=====
Omnibus:                    659.818   Durbin-Watson:           1.982
Prob(Omnibus):              0.000   Jarque-Bera (JB):        1104.314
Skew:                      0.679   Prob(JB):                1.59e-240
Kurtosis:                   4.393   Cond. No.                 1.83
=====
```

Notes:

[1] Standard Errors assume that the covariance matrix of the errors is correctly specified.

```
Variable   VIF
3   var_9  1.41
2   var_3  1.23
1   var_2  1.17
0   const  1.00
```

```
In [23]: # ===== Model Evaluation for Linear Regression =====

evaluate_model(model_ols, X_train_const, y_train, X_val_data[x_var], y_val, X_test_

# Calculate residuals
y_train_pred = model_ols.predict(X_train_const)
residuals = y_train - y_train_pred

# Plot residuals
plt.figure(figsize=(12, 5))

# Residuals distribution
plt.subplot(1, 2, 1)
```

```

sns.histplot(residuals, kde=True)
plt.title('Residuals Distribution')
plt.xlabel('Residuals')

# Q-Q Plot (to check normality)
plt.subplot(1, 2, 2)
import scipy.stats as stats
stats.probplot(residuals, dist="norm", plot=plt)
plt.title('Q-Q Plot of Residuals')

```

=== Linear Regression ===

Train R^2 : 0.7717

Val R^2 : 0.7662

Test R^2 : 0.7671

Train MAE: 5.8571

Val MAE: 6.1640

Test MAE: 5.9722

Train RMSE: 7.5807

Val RMSE: 7.9755

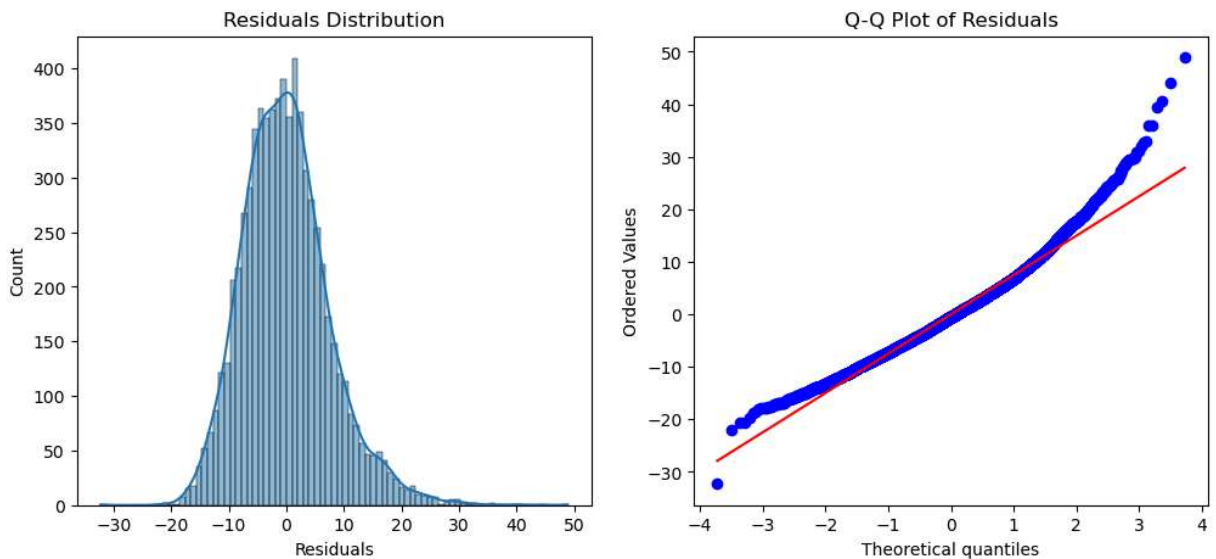
Test RMSE: 7.8355

0.9882080945148274 7.4950445461306945 -8.249772371934561e-15

Mean of Residuals: -0.0

Std of Residuals: 7.5813

Out[23]: Text(0.5, 1.0, 'Q-Q Plot of Residuals')



In [25]: # ===== Linear Regression Model with Interaction Terms =====

```
x_var=['var_2', 'var_3', 'var_9', 'var_10', 'var_3*var_3', 'var_2*var_2']
```

```
# Add constant (intercept)
```

```
X_train_const = sm.add_constant(X_train_data[x_var])
```

```
# Fit model
```

```
model_ols2 = sm.OLS(y_train, X_train_const).fit()
```

```
# Show summary
```

```
print(model_ols2.summary())
```

```
# Show summary VIF
get_VIF (X_train_const)
```

```

=====
                        OLS Regression Results
=====
Dep. Variable:          y      R-squared:                0.881
Model:                  OLS    Adj. R-squared:           0.881
Method:                 Least Squares    F-statistic:      8620.
Date:                  Tue, 09 Jun 2026    Prob (F-statistic): 0.00
Time:                  11:31:37    Log-Likelihood:    -21834.
No. Observations:      7000    AIC:               4.368e+04
Df Residuals:          6993    BIC:               4.373e+04
Df Model:              6
Covariance Type:       nonrobust
=====

```

	coef	std err	t	P> t	[0.025	0.975]
const	9.0736	0.168	53.885	0.000	8.744	9.404
var_2	10.7419	0.071	151.459	0.000	10.603	10.881
var_3	-6.8380	0.073	-94.196	0.000	-6.980	-6.696
var_9	7.4178	0.078	95.186	0.000	7.265	7.571
var_10	-0.6008	0.047	-12.876	0.000	-0.692	-0.509
var_3*var_3	1.2691	0.046	27.457	0.000	1.179	1.360
var_2*var_2	3.4897	0.047	74.179	0.000	3.397	3.582

```

=====
Omnibus:              333.037    Durbin-Watson:          1.963
Prob(Omnibus):        0.000    Jarque-Bera (JB):       618.638
Skew:                 -0.365    Prob(JB):               4.62e-135
Kurtosis:             4.261    Cond. No.:               9.91
=====

```

Notes:

[1] Standard Errors assume that the covariance matrix of the errors is correctly specified.

	Variable	VIF
0	const	6.61
3	var_9	1.42
2	var_3	1.23
1	var_2	1.17
6	var_2*var_2	1.00
4	var_10	1.00
5	var_3*var_3	1.00

```
In [27]: # ===== Linear Regression 2 Model Evaluation =====

evaluate_model(model_ols2, X_train_const, y_train, X_val_data[x_var], y_val, X_test

# Calculate residuals
y_train_pred = model_ols2.predict(X_train_const)
residuals = y_train - y_train_pred

# Plot residuals
plt.figure(figsize=(12, 5))

# Residuals distribution
plt.subplot(1, 2, 1)
```

```

sns.histplot(residuals, kde=True)
plt.title('Residuals Distribution')
plt.xlabel('Residuals')

# Q-Q Plot (to check normality)
plt.subplot(1, 2, 2)
import scipy.stats as stats
stats.probplot(residuals, dist="norm", plot=plt)
plt.title('Q-Q Plot of Residuals')

```

=== Linear Regression 2 ===

Train R^2 : 0.8809

Val R^2 : 0.8782

Test R^2 : 0.8935

Train MAE: 4.3386

Val MAE: 4.5592

Test MAE: 4.2578

Train RMSE: 5.4750

Val RMSE: 5.7560

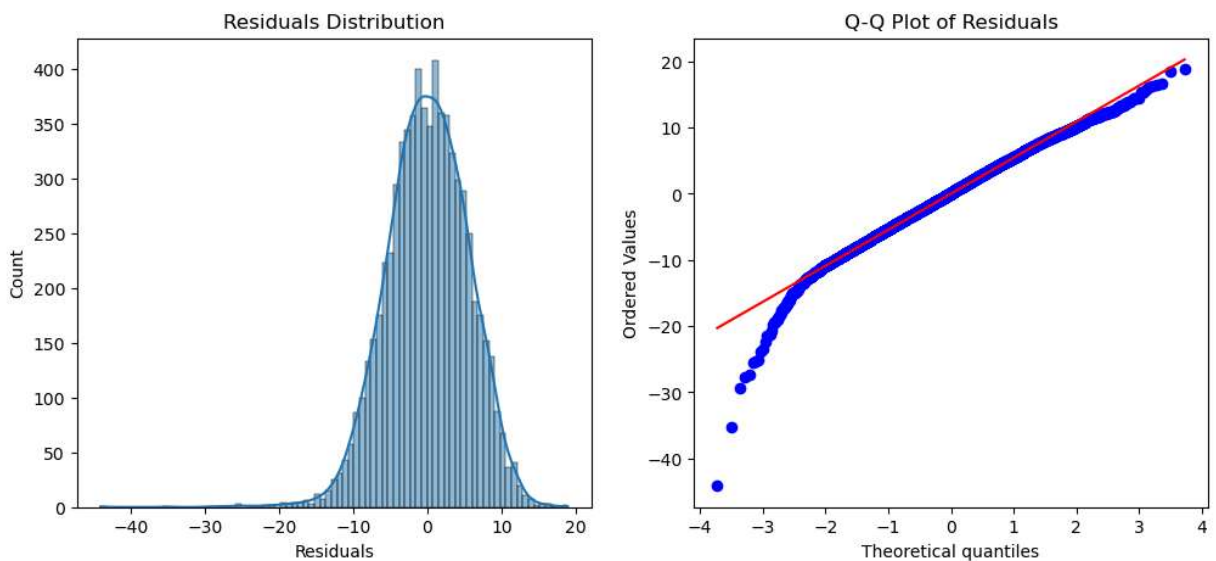
Test RMSE: 5.2978

0.9947615221598559 5.44905436937417 -1.525989895388377e-14

Mean of Residuals: -0.0

Std of Residuals: 5.4754

Out[27]: Text(0.5, 1.0, 'Q-Q Plot of Residuals')



In [29]: # ===== Linear Regression Model with Interaction Terms and Change Var10

```
x_var=['var_2', 'var_3', 'var_9', 'var_3*var_3', 'var_2*var_2', 'var_10_4', 'var_10
```

```
# Add constant (intercept)
```

```
X_train_const = sm.add_constant(X_train_data[x_var])
```

```
# Fit model
```

```
model_ols3 = sm.OLS(y_train, X_train_const).fit()
```

```
# Show summary
```

```
print(model_ols3.summary())
```

```
# Show summary VIF
get_VIF (X_train_const)
```

```

                                OLS Regression Results
=====
Dep. Variable:                  y      R-squared:                  0.906
Model:                          OLS    Adj. R-squared:             0.906
Method:                        Least Squares  F-statistic:                8443.
Date:                          Tue, 09 Jun 2026  Prob (F-statistic):      0.00
Time:                          11:31:54  Log-Likelihood:             -20998.
No. Observations:              7000    AIC:                       4.201e+04
Df Residuals:                  6991    BIC:                       4.208e+04
Df Model:                      8
Covariance Type:               nonrobust
=====
                                coef    std err          t      P>|t|      [0.025    0.975]
-----
const                7.2068      0.110     65.340     0.000      6.991      7.423
var_2               10.7752      0.063    171.183     0.000     10.652     10.899
var_3               -6.8305      0.064   -106.017     0.000     -6.957     -6.704
var_9                7.4654      0.069    107.930     0.000      7.330      7.601
var_3*var_3          1.2648      0.041     30.824     0.000      1.184      1.345
var_2*var_2          3.4962      0.042     83.745     0.000      3.414      3.578
var_10_4            -4.5975      0.158    -29.010     0.000     -4.908     -4.287
var_10_2             3.3486      0.160     20.898     0.000      3.035      3.663
var_10_3             1.6159      0.158     10.203     0.000      1.305      1.926
=====
Omnibus:                 556.792    Durbin-Watson:             1.977
Prob(Omnibus):           0.000    Jarque-Bera (JB):          1582.261
Skew:                    -0.432    Prob(JB):                  0.00
Kurtosis:                5.163    Cond. No.                  8.18
=====

```

Notes:

[1] Standard Errors assume that the covariance matrix of the errors is correctly specified.

```

Variable  VIF
0      const  3.60
3      var_9  1.42
2      var_3  1.23
6  var_10_4  1.21
8  var_10_3  1.21
7  var_10_2  1.21
1      var_2  1.17
5  var_2*var_2  1.00
4  var_3*var_3  1.00

```

```

In [31]: # ===== Linear Regression 3 Model Evaluation =====

evaluate_model(model_ols3, X_train_const, y_train, X_val_data[x_var], y_val, X_test

# Calculate residuals
y_train_pred = model_ols3.predict(X_train_const)
residuals = y_train - y_train_pred

# Plot residuals

```

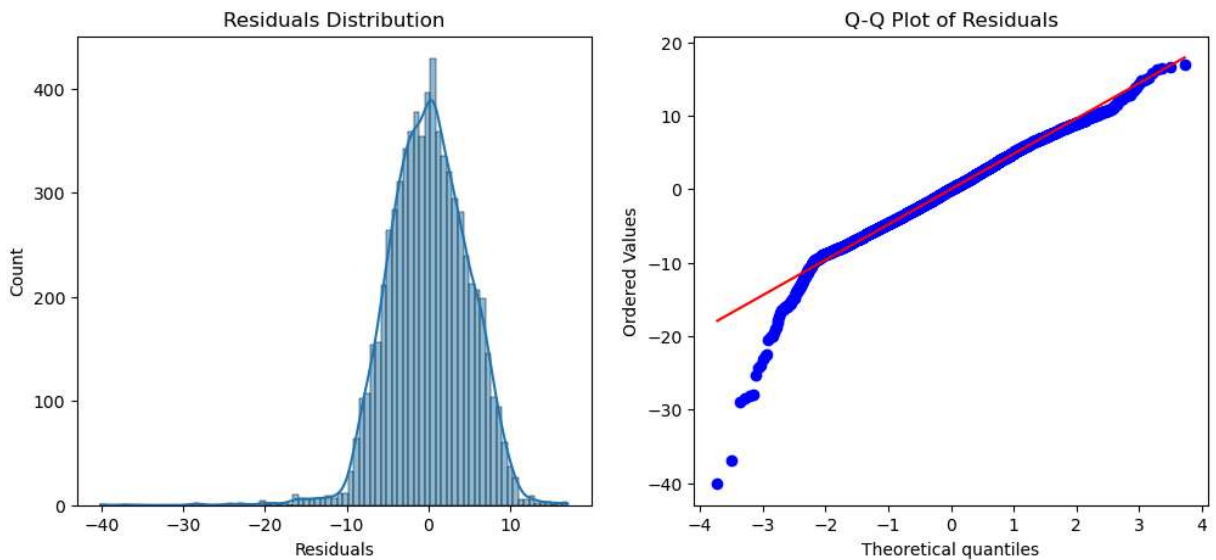
```
plt.figure(figsize=(12, 5))

# Residuals distribution
plt.subplot(1, 2, 1)
sns.histplot(residuals, kde=True)
plt.title('Residuals Distribution')
plt.xlabel('Residuals')

# Q-Q Plot (to check normality)
plt.subplot(1, 2, 2)
import scipy.stats as stats
stats.probplot(residuals, dist="norm", plot=plt)
plt.title('Q-Q Plot of Residuals')
```

```
=== Linear Regression 3 ===
Train R² : 0.9062
Val R² : 0.9066
Test R² : 0.9179
Train MAE: 3.8198
Val MAE: 3.9422
Test MAE: 3.7262
Train RMSE: 4.8586
Val RMSE: 5.0409
Test RMSE: 4.6535
0.9907819145126119 4.81619802408749 7.949477675916635e-15
Mean of Residuals: 0.0
Std of Residuals: 4.859
```

Out[31]: Text(0.5, 1.0, 'Q-Q Plot of Residuals')



```
In [33]: # ===== Linear Robust Regression Model with Interaction Terms and Chang

x_var=['var_2', 'var_3', 'var_9', 'var_3*var_3', 'var_2*var_2', 'var_10_4', 'var_10
# Add constant (intercept)
X_train_const = sm.add_constant(X_train_data[x_var])

# Fit model
model_olsHc3 = sm.OLS(y_train, X_train_const).fit(cov_type='HC3')

# Show summary
```

```
print(model_olsHc3.summary())
```

```
# Show summary VIF  
get_VIF (X_train_const)
```

OLS Regression Results

```
=====
```

Dep. Variable:	y	R-squared:	0.906
Model:	OLS	Adj. R-squared:	0.906
Method:	Least Squares	F-statistic:	7102.
Date:	Tue, 09 Jun 2026	Prob (F-statistic):	0.00
Time:	11:32:04	Log-Likelihood:	-20998.
No. Observations:	7000	AIC:	4.201e+04
Df Residuals:	6991	BIC:	4.208e+04
Df Model:	8		
Covariance Type:	HC3		

```
=====
```

	coef	std err	z	P> z	[0.025	0.975]
-----	-----	-----	-----	-----	-----	-----
const	7.2068	0.132	54.404	0.000	6.947	7.466
var_2	10.7752	0.067	160.972	0.000	10.644	10.906
var_3	-6.8305	0.080	-85.850	0.000	-6.986	-6.675
var_9	7.4654	0.083	89.532	0.000	7.302	7.629
var_3*var_3	1.2648	0.087	14.509	0.000	1.094	1.436
var_2*var_2	3.4962	0.059	59.421	0.000	3.381	3.611
var_10_4	-4.5975	0.159	-28.900	0.000	-4.909	-4.286
var_10_2	3.3486	0.158	21.186	0.000	3.039	3.658
var_10_3	1.6159	0.161	10.018	0.000	1.300	1.932
-----	-----	-----	-----	-----	-----	-----

Omnibus:	556.792	Durbin-Watson:	1.977
Prob(Omnibus):	0.000	Jarque-Bera (JB):	1582.261
Skew:	-0.432	Prob(JB):	0.00
Kurtosis:	5.163	Cond. No.	8.18

```
=====
```

Notes:

[1] Standard Errors are heteroscedasticity robust (HC3)

	Variable	VIF
0	const	3.60
3	var_9	1.42
2	var_3	1.23
6	var_10_4	1.21
8	var_10_3	1.21
7	var_10_2	1.21
1	var_2	1.17
5	var_2*var_2	1.00
4	var_3*var_3	1.00

```
In [35]: # ===== Linear Robust Regression Model Evaluation =====  
  
evaluate_model(model_olsHc3, X_train_const, y_train, X_val_data[x_var], y_val, X_te  
  
# Calculate residuals  
y_train_pred = model_olsHc3.predict(X_train_const)  
residuals = y_train - y_train_pred
```

```

# Plot residuals
plt.figure(figsize=(12, 5))

# Residuals distribution
plt.subplot(1, 2, 1)
sns.histplot(residuals, kde=True)
plt.title('Residuals Distribution')
plt.xlabel('Residuals')

# Q-Q Plot (to check normality)
plt.subplot(1, 2, 2)
import scipy.stats as stats
stats.probplot(residuals, dist="norm", plot=plt)
plt.title('Q-Q Plot of Residuals')

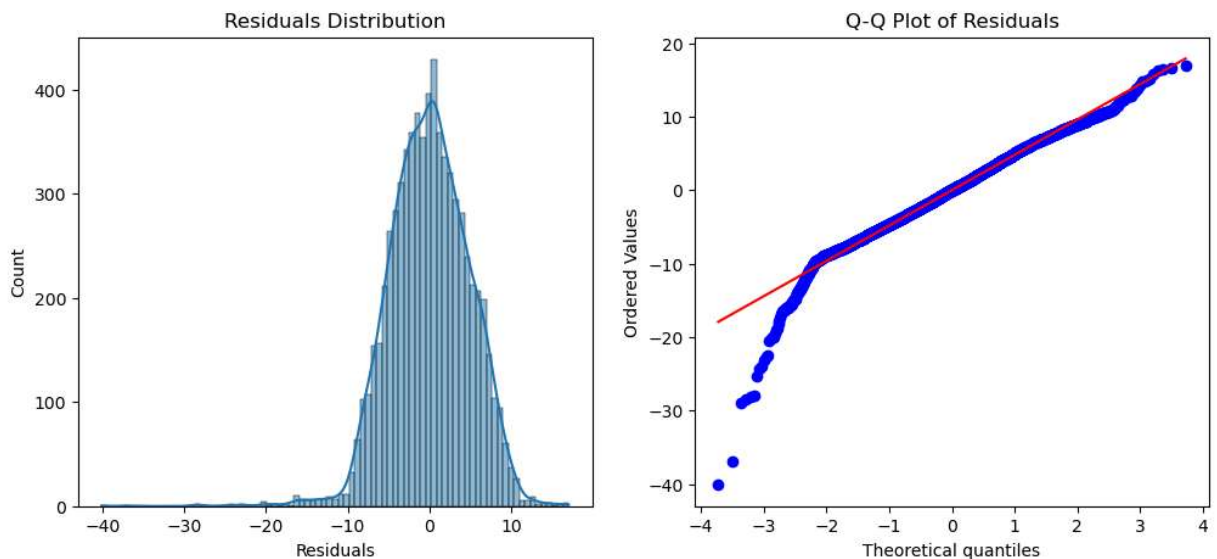
```

```

=== Linear Robust Regression ===
Train R² : 0.9062
Val R² : 0.9066
Test R² : 0.9179
Train MAE: 3.8198
Val MAE: 3.9422
Test MAE: 3.7262
Train RMSE: 4.8586
Val RMSE: 5.0409
Test RMSE: 4.6535
0.9907819145126119 4.81619802408749 7.949477675916635e-15
Mean of Residuals: 0.0
Std of Residuals: 4.859

```

Out[35]: Text(0.5, 1.0, 'Q-Q Plot of Residuals')



```

In [37]: # ===== Linear Regression Model with Piecewise Vars Added (Synthetic D

x_var=['var_2*var_2', 'var_2', 'var_10_4', 'var_10_2', 'var_10_3', 'var3_0_to_200k'

# Add constant (intercept)
X_train_const = sm.add_constant(X_train_data[x_var])

# Fit model
model_ols4 = sm.OLS(y_train, X_train_const).fit()

```

```
# Show summary
print(model_ols4.summary())

# Show summary VIF
get_VIF (X_train_const)
```

```

=====
                        OLS Regression Results
=====
Dep. Variable:          y      R-squared:          0.998
Model:                  OLS    Adj. R-squared:       0.998
Method:                 Least Squares  F-statistic:    5.088e+05
Date:                   Tue, 09 Jun 2026  Prob (F-statistic): 0.00
Time:                   11:32:19  Log-Likelihood: -7457.8
No. Observations:       7000     AIC:            1.493e+04
Df Residuals:           6992     BIC:            1.499e+04
Df Model:                7
Covariance Type:        nonrobust
=====
=

```

	coef	std err	t	P> t	[0.025	0.97
5]						
-						
const	11.3998	0.027	425.511	0.000	11.347	11.45
2						
var_2*var_2	3.5287	0.006	584.968	0.000	3.517	3.54
1						
var_2	10.7179	0.009	1203.140	0.000	10.700	10.73
5						
var_10_4	-4.5516	0.023	-198.783	0.000	-4.596	-4.50
7						
var_10_2	3.4780	0.023	150.207	0.000	3.433	3.52
3						
var_10_3	1.4866	0.023	64.960	0.000	1.442	1.53
1						
var3_0_to_200k	-20.0091	0.023	-860.964	0.000	-20.055	-19.96
4						
var9_100k_to_200k	19.8284	0.023	872.910	0.000	19.784	19.87
3						

```

=====
Omnibus:                8732.100  Durbin-Watson:          1.946
Prob(Omnibus):           0.000    Jarque-Bera (JB):        3600004.436
Skew:                    -6.386    Prob(JB):                0.00
Kurtosis:                113.362    Cond. No.:               8.77
=====

```

Notes:

[1] Standard Errors assume that the covariance matrix of the errors is correctly specified.

	Variable	VIF
0	const	10.18
7	var9_100k_to_200k	1.27
3	var_10_4	1.21
5	var_10_3	1.21
4	var_10_2	1.21
6	var3_0_to_200k	1.14
2	var_2	1.13
1	var_2*var_2	1.00

```

In [39]: # ===== Linear Regression 4 Model Evaluation =====

evaluate_model(model_ols4, X_train_const, y_train, X_val_data[x_var], y_val, X_test

```

```

# Calculate residuals
y_train_pred = model_ols4.predict(X_train_const)
residuals = y_train - y_train_pred

# Plot residuals
plt.figure(figsize=(12, 5))

# Residuals distribution
plt.subplot(1, 2, 1)
sns.histplot(residuals, kde=True)
plt.title('Residuals Distribution')
plt.xlabel('Residuals')

# Q-Q Plot (to check normality)
plt.subplot(1, 2, 2)
import scipy.stats as stats
stats.probplot(residuals, dist="norm", plot=plt)
plt.title('Q-Q Plot of Residuals')

```

=== Linear Regression 4 ===

Train R^2 : 0.9980

Val R^2 : 0.9986

Test R^2 : 0.9972

Train MAE: 0.4282

Val MAE: 0.4075

Test MAE: 0.4455

Train RMSE: 0.7022

Val RMSE: 0.6131

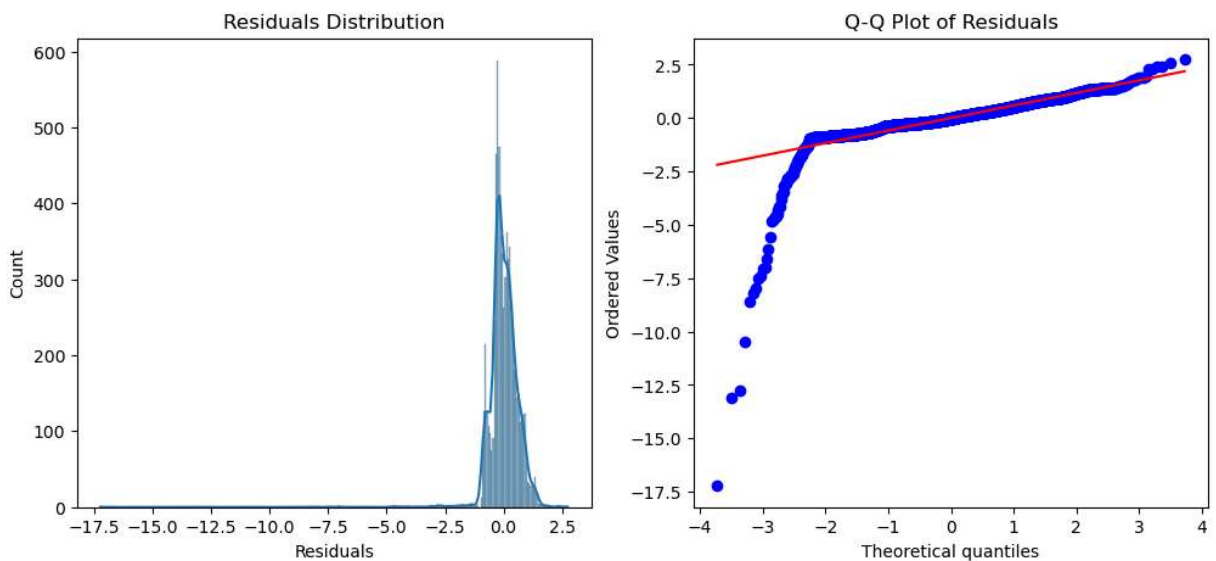
Test RMSE: 0.8525

0.8375638950016601 0.5884303172514378 5.1531410700653735e-15

Mean of Residuals: 0.0

Std of Residuals: 0.7023

Out[39]: Text(0.5, 1.0, 'Q-Q Plot of Residuals')



In []:

```
In [41]: # ===== Model Evaluation Function For Random Forest and XGBoost Models

def evaluate_model(model, X_train, y_train, X_val, y_val, X_test, y_test, name="Mod

    y_train_pred = model.predict(X_train)
    y_val_pred = model.predict(X_val)
    y_test_pred = model.predict(X_test)

    print(f"\n=== {name} ===")
    print(f"Train R² : {r2_score(y_train, y_train_pred):.4f}")
    print(f"Val R² : {r2_score(y_val, y_val_pred):.4f}")
    print(f"Test R² : {r2_score(y_test, y_test_pred):.4f}")
    print(f"Train MAE: {mean_absolute_error(y_train, y_train_pred):.4f}")
    print(f"Val MAE: {mean_absolute_error(y_val, y_val_pred):.4f}")
    print(f"Test MAE: {mean_absolute_error(y_test, y_test_pred):.4f}")
    print(f"Train RMSE: {np.sqrt(mean_squared_error(y_train, y_train_pred)):.4f}")
    print(f"Val RMSE: {np.sqrt(mean_squared_error(y_val, y_val_pred)):.4f}")
    print(f"Test RMSE: {np.sqrt(mean_squared_error(y_test, y_test_pred)):.4f}")
```

```
In [43]: # ===== Random Forest Model =====

x_var=['var_2', 'var_3', 'var_9', 'var_10']

model_rf = RandomForestRegressor(
    n_estimators=300,
    max_depth=10,
    min_samples_leaf=30,
    min_samples_split=60,
    max_features="sqrt",
    oob_score=True,
    bootstrap=True,
    random_state=63,
    n_jobs=-1)
model_rf.fit(X_train_data[x_var], y_train)

depths = [t.tree_.max_depth for t in model_rf.estimators_]
leaves = [t.tree_.n_leaves for t in model_rf.estimators_]

print("Avg depth:", sum(depths)/len(depths))
print("Avg leaves:", sum(leaves)/len(leaves))
print("oob_score:", model_rf.oob_score_)
```

```
Avg depth: 10.0
Avg leaves: 111.2
oob_score: 0.9622525255276725
```

```
In [45]: # ===== Model Features Checking =====

print (model_rf.feature_importances_)

importance = pd.Series(model_rf.feature_importances_, index=X_train_data[x_var].col
importance.sort_values(ascending=False)
```

```
[0.4087391  0.37584513 0.20393805 0.01147773]
```

```
Out[45]: var_2      0.408739
         var_3      0.375845
         var_9      0.203938
         var_10     0.011478
         dtype: float64
```

```
In [47]: # ===== Random Forest Model Evaluation =====

evaluate_model(model_rf, X_train_data[x_var], y_train, X_val_data[x_var], y_val, X_

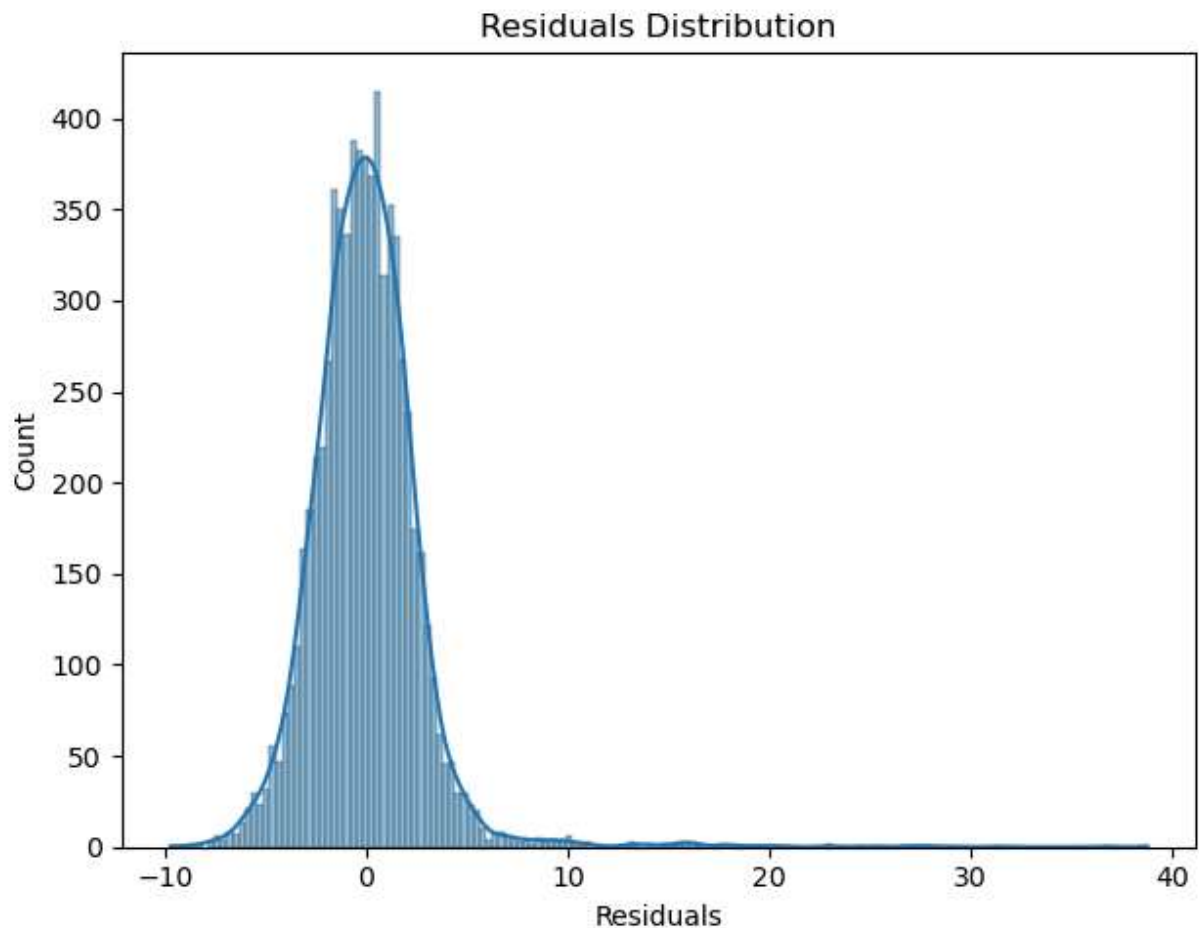
# Calculate residuals
y_train_pred = model_rf.predict(X_train_data[x_var])
residuals = y_train - y_train_pred

# Plot residuals
plt.figure(figsize=(12, 5))

# Residuals distribution
plt.subplot(1, 2, 1)
sns.histplot(residuals, kde=True)
plt.title('Residuals Distribution')
plt.xlabel('Residuals')

plt.tight_layout()
plt.show()

=== Random Forests ===
Train R² : 0.9682
Val R² : 0.9546
Test R² : 0.9512
Train MAE: 1.9013
Val MAE: 2.1525
Test MAE: 2.1941
Train RMSE: 2.8285
Val RMSE: 3.5142
Test RMSE: 3.5883
```



In [49]: # ===== XGBoost Model =====

```
x_var=['var_2', 'var_3', 'var_9', 'var_10']
```

```
model_xgb = XGBRegressor(  
    n_estimators=300,  
    learning_rate=0.03,  
    max_depth=3,  
    min_child_weight=30,  
    subsample=0.7,  
    colsample_bytree=0.7,  
    reg_alpha=1,  
    reg_lambda=0.5,  
    random_state=63,  
    early_stopping_rounds=20  
)
```

```
model_xgb.fit(  
    X_train_data[x_var],  
    y_train,  
    eval_set=[(X_val_data[x_var], y_val)],  
    verbose=False  
)
```

Out[49]:

▼ XGBRegressor ⓘ ?

► Parameters

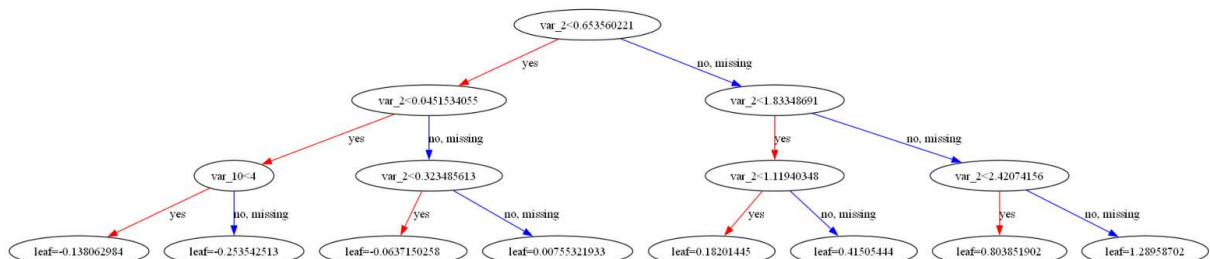
In [51]: # ===== Model Trees Checking =====

```
trees = (model_xgb.get_booster()).get_dump()
print(len(trees))
print(trees[0])
```

```
fig, ax = plt.subplots(figsize=(20,10))
plot_tree(model_xgb, tree_idx=4, ax=ax)
plt.show()
```

300

```
0:[var_3<-0.161508873] yes=1,no=2,missing=2
  1:[var_3<-0.751046777] yes=3,no=4,missing=4
    3:[var_9<-0.0488123856] yes=7,no=8,missing=8
      7:leaf=0.279331774
      8:leaf=0.527481973
    4:[var_9<0.00519318972] yes=9,no=10,missing=10
      9:leaf=-0.0261040423
      10:leaf=0.261920869
  2:[var_9<0.0928892121] yes=5,no=6,missing=6
    5:[var_9<-1.64866984] yes=11,no=12,missing=12
      11:leaf=-0.127805918
      12:leaf=-0.368120909
    6:[var_3<-0.0181094278] yes=13,no=14,missing=14
      13:leaf=0.0954357311
      14:leaf=-0.0734186545
```



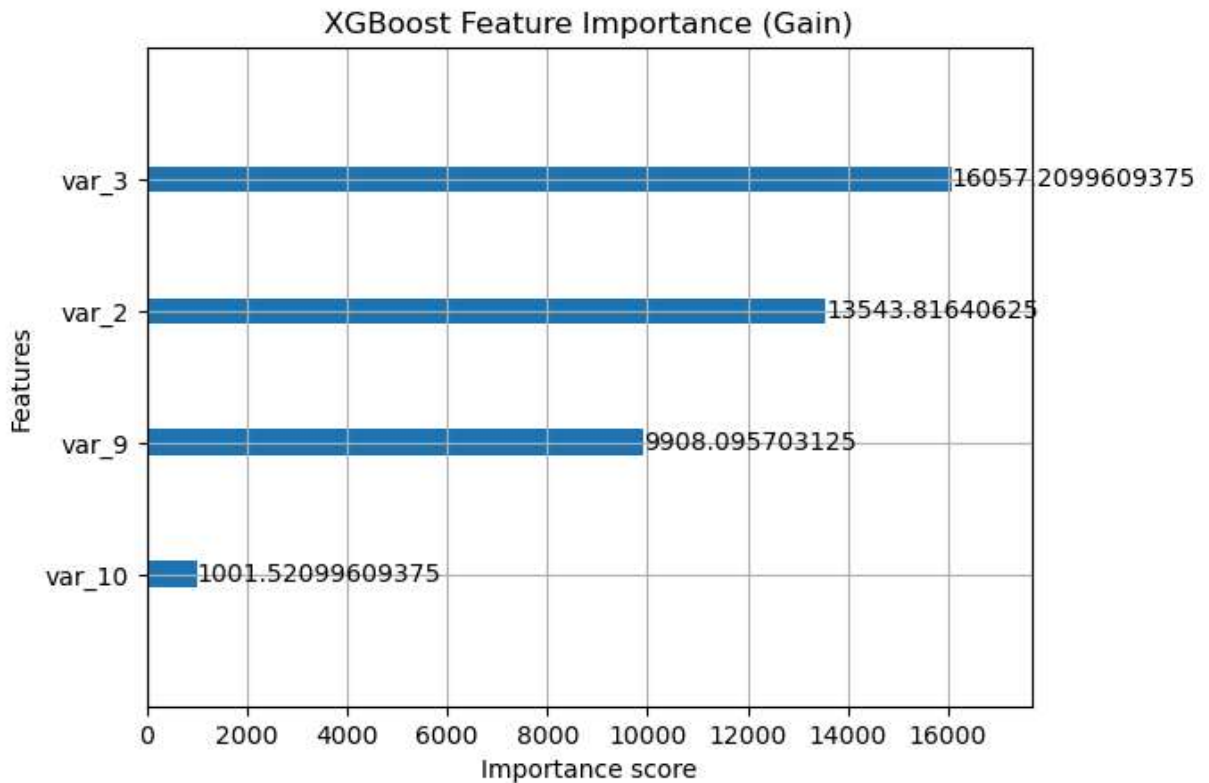
In [53]: # ===== Model Features Checking =====

```
print(model_xgb.feature_importances_)
```

```
plt.figure(figsize=(10, 8))
plot_importance(model_xgb, importance_type='gain', max_num_features=10)
plt.title('XGBoost Feature Importance (Gain)')
plt.show()
```

[0.33432734 0.39637014 0.24458006 0.02472242]

<Figure size 1000x800 with 0 Axes>



```
In [55]: # ===== Model Features Checking Continue =====

explainer = shap.Explainer(model_xgb.predict, X_test_data[x_var])
shap_values = explainer(X_test_data[x_var])

importance = np.abs(shap_values.values).mean(axis=0)
feature_importance = pd.Series(importance, index=X_test_data[x_var].columns)

pd.set_option('display.max_rows', None)
print(feature_importance.sort_values(ascending=False))
```

```
ExactExplainer explainer: 1501it [00:12, 54.18it/s]
var_2      8.848294
var_9      7.195034
var_3      7.013992
var_10     1.695131
dtype: float64
```

```
In [57]: # ===== XGBoost Model Evaluation =====

evaluate_model(model_xgb, X_train_data[x_var], y_train, X_val_data[x_var], y_val, X

# Calculate residuals
y_train_pred = model_xgb.predict(X_train_data[x_var])
residuals = y_train - y_train_pred

# Plot residuals
plt.figure(figsize=(12, 5))

# Residuals distribution
plt.subplot(1, 2, 1)
```

```
sns.histplot(residuals, kde=True)
plt.title('Residuals Distribution')
plt.xlabel('Residuals')

plt.tight_layout()
plt.show()
```

=== XGBooster Model ===

Train R^2 : 0.9948

Val R^2 : 0.9909

Test R^2 : 0.9914

Train MAE: 0.7460

Val MAE: 0.8754

Test MAE: 0.8554

Train RMSE: 1.1464

Val RMSE: 1.5741

Test RMSE: 1.5099

